

Приватний вищий навчальний заклад
«ЄВРОПЕЙСЬКИЙ УНІВЕРСИТЕТ»

**КОМП'ЮТЕРНА ПРОГРАМА
«ТЕЛЕГРАМ-БОТ ДЛЯ ІНФОРМУВАННЯ СТУДЕНТІВ
ПРО ПОТОЧНІ ОЦІНКИ»**

Тимошенко Олена Іванівна
Чопоров Сергій Вікторович
Козинець Анатолій Олександрович
Бабінчук Олена Петрівна

Київ, 2026

ЗМІСТ

1.	ВИХІДНІ ПРОГРАМНІ КОДИ	3
1.1.	README.md.....	3
1.2.	requirements.txt	4
1.3.	eu_grades_bot/__init__.py.....	5
1.4.	eu_grades_bot/__main__.py	5
1.5.	eu_grades_bot/bot.py	5
1.6.	eu_grades_bot/config.py	15
1.7.	eu_grades_bot/dates.py	16
1.8.	eu_grades_bot/drive.py	16
1.9.	eu_grades_bot/formatting.py	22
1.10.	eu_grades_bot/grades.py	26
1.11.	eu_grades_bot/storage.py	30

1. ВИХІДНІ ПРОГРАМНІ КОДИ

1.1. README.md

Телеграм-бот для інформування студентів про поточні оцінки

Бот читає таблиці з оцінками з локальної директорії або Google Drive і дає студентам доступ лише до власних записів після авторизації email-адресою.

Можливості

- `/start` - авторизація або перегляд прив'язаної пошти.
- `/stop` - припинити діалог із ботом і видалити прив'язану email-адресу.
- `/email student@example.com` - змінити email.
- `/refresh` - примусово перечитати таблиці з Google Drive або локальної директорії.
- `/subjects` - перелік дисциплін, де знайдено email студента.
- `/grades` - усі власні оцінки та пропуски з групуванням за датами й середнім балом за кожною дисципліною.
- `/today` - записи за поточний день.
- `/week` - записи за поточний тиждень.
- `/lastweek` - записи за попередній тиждень.
- `/subject частина назви` - записи з конкретної дисципліни.

Після `/start`, `/help` та авторизації бот показує inline-меню з кнопками `/grades`, `/today`, `/week`.

Для `/today` і щоденного зведення дата друкується один раз на початку повідомлення. Оцінки виводяться як `Назва дисципліни - оцінка`, пропуски - як перелік назв дисциплін, а за наявності пропусків додається їх загальна кількість.

Для `/week`, `/lastweek` і недільного зведення записи групуються за датами. Наприкінці повідомлення додається статистика пропусків, загальна кількість оцінок і кількість оцінок за рівнями: `відмінно`, `добре`, `задовільно`, `незадовільно`.

У `GRADES_SOURCE=drive` звичайні звіти й inline-кнопки використовують кеш для швидшої відповіді. Примусове перечитування Google Drive виконується для `/refresh`, `/subjects` і один раз на початку планової денної або тижневої розсилки.

Кожен будній день о 19:00 бот надсилає денне зведення. Щонеділі о 10:00 бот надсилає зведення за поточний тиждень.

Telegram Bot API не підтримує довільний колір тексту в повідомленнях, тому бот позначає пропуски та оцінки 1-3 червоним маркером , а оцінки 10-12 зеленим маркером . Назви дисциплін форматуються жирним.

Налаштування

1. Створіть віртуальне середовище та встановіть залежності:

```
```powershell
py -m venv .venv
.venv\Scripts\pip install -r requirements.txt
```
```

2. Скопіюйте `.env.example` у `.env` і заповніть значення.

Для локальної синхронізованої папки Google Drive:

```
```dotenv
TELEGRAM_BOT_TOKEN=123456:telegram-token
GRADES_SOURCE=local
GRADES_LOCAL_DIR=C:\path\to\drive\folder
GOOGLE_APPLICATION_CREDENTIALS=C:\path\to\service-account.json
```
```

Якщо в локальній директорії Google Drive файли мають вигляд `Назва дисципліни - Викладач.gsheets`, вони є ярликами Google Sheets, а не самими таблицями. Для таких файлів потрібен `GOOGLE\_APPLICATION\_CREDENTIALS`: бот витягне id таблиці з `.gsheet` і експортує Google Sheets у `.xlsx`.

Для Google Drive API:

```
```dotenv
TELEGRAM_BOT_TOKEN=123456:telegram-token
GRADES_SOURCE=drive
GRADES_DRIVE_FOLDER_ID=google-drive-folder-id
GOOGLE_APPLICATION_CREDENTIALS=C:\path\to\service-account.json
```
```

Service account має мати доступ до директорії з таблицями. `GRADES\_DRIVE\_FOLDER\_ID` може бути raw ID директорії або повним URL директорії. У Drive-режимі бот читає native Google Sheets, `.xlsx` файли, shortcuts на Google Sheets і файли з назвою `\*.gsheet`, екпортує Google Sheets у `.xlsx` для парсингу.

3. Запустіть бота:

```
```powershell
.venv\Scripts\python -m eu_grades_bot
```
```

Формат таблиць

Назва файлу має відповідати схемі `Назва дисципліни - Прізвище І.П..xlsx`. Якщо файл є Google Sheets, бот експортує його як `.xlsx`.

У кожному аркуші:

- перший рядок містить заголовки;
- одна з колонок має заголовок `Електронна пошта` або `email`;
- колонки з датами містять оцінки `1-12` або пропуск `в`; у повідомленнях бота пропуск відображається як `відсутній`;
- кожен аркуш вважається окремою академічною групою.

1.2. requirements.txt

```
openpyxl>=3.1,<4
python-dotenv>=1.0,<2
python-telegram-bot[job-queue]>=21,<23
google-api-python-client>=2.120,<3
google-auth>=2.29,<3
```

1.3. eu_grades_bot/__init__.py

```
"""Telegram bot for student grade spreadsheets."""
```

```
__all__ = ["__version__"]
```

```
__version__ = "0.1.0"
```

1.4. eu_grades_bot/__main__.py

```
from .bot import main
```

```
if __name__ == "__main__":
    main()
```

1.5. eu_grades_bot/bot.py

```
from __future__ import annotations
```

```
import logging
```

```
from datetime import time
```

```
from telegram import InlineKeyboardButton, InlineKeyboardMarkup, Update
```

```
from telegram.constants import ParseMode
```

```
from telegram.ext import (
```

```
    Application,
    ApplicationBuilder,
    CallbackQueryHandler,
    CommandHandler,
    ContextTypes,
    MessageHandler,
    filters,
```

```
)
```

```
from .config import Settings, load_settings
```

```
from .dates import current_week_bounds, date_range_filter, previous_week_bounds,
today_in
```

```
from .drive import DriveWorkbookProvider, LocalWorkbookProvider
```

```
from .formatting import (
```

```
    format_entries,
    format_period_entries_by_date,
    format_subjects,
    format_today_entries,
    split_telegram_message,
```

```
)
```

```
from .grades import GradesRepository, is_valid_email, normalize_email
```

```
from .storage import UserStorage
```

```
logging.basicConfig(
```

```
    level=logging.INFO,
```

```
    format="%(asctime)s %(levelname)s %(name)s: %(message)s",
```

```
)
```

```
logger = logging.getLogger(__name__)
```

```

HELP_TEXT = """Команди:
/start - авторизація email
/stop - припинити діалог і видалити email
/email student@example.com - змінити email
/refresh - перечитати таблиці з Drive/папки
/subjects - перелік дисциплін
/grades - усі оцінки та пропуски
/grades today - за сьогодні
/grades week - за поточний тиждень
/grades lastweek - за попередній тиждень
/grades назва дисципліни - з конкретної дисципліни
/today - за сьогодні
/week - за поточний тиждень
/lastweek - за попередній тиждень
/subject назва дисципліни - з конкретної дисципліни"""

```

```

REPORT_MENU_TEXT = "Оберіть звіт:"
REPORT_CALLBACK_PREFIX = "report:"

```

```

class BotServices:
    def __init__(self, settings: Settings):
        self.settings = settings
        self.storage = UserStorage(settings.data_dir / "users.json")
        self.repository = GradesRepository(
            provider=create_provider(settings),
            cache_ttl_seconds=settings.cache_ttl_seconds,
        )

def create_provider(settings: Settings):
    if settings.grades_source == "local":
        return LocalWorkbookProvider(
            settings.local_grades_dir,
            credentials_file=settings.google_credentials_file,
            cache_dir=settings.data_dir / "local_gsheets_cache",
        )
    if not settings.drive_folder_id:
        raise ValueError("GRADES_DRIVE_FOLDER_ID is required for GRADES_SOURCE=drive.")
    if not settings.google_credentials_file:
        raise ValueError("GOOGLE_APPLICATION_CREDENTIALS is required for GRADES_SOURCE=drive.")
    return DriveWorkbookProvider(
        folder_id=settings.drive_folder_id,
        credentials_file=settings.google_credentials_file,
        cache_dir=settings.data_dir / "drive_cache",
    )

def report_menu_keyboard() -> InlineKeyboardMarkup:
    return InlineKeyboardMarkup(
        [
            [InlineKeyboardButton("/grades",
callback_data=f"{REPORT_CALLBACK_PREFIX}grades")],
        ]
    )

```

```

        InlineKeyboardButton("/today",
callback_data=f"{REPORT_CALLBACK_PREFIX}today"),
        InlineKeyboardButton("/week",
callback_data=f"{REPORT_CALLBACK_PREFIX}week"),
    ],
]
)

def main() -> None:
    settings = load_settings()
    if not settings.telegram_bot_token:
        raise ValueError("TELEGRAM_BOT_TOKEN is required.")

    services = BotServices(settings)
    application = build_application(settings, services)
    application.run_polling(allowed_updates=Update.ALL_TYPES)

def build_application(settings: Settings, services: BotServices) -> Application:
    application = ApplicationBuilder().token(settings.telegram_bot_token).build()
    application.bot_data["services"] = services

    application.add_handler(CommandHandler("start", start))
    application.add_handler(CommandHandler("stop", stop_command))
    application.add_handler(CommandHandler("help", help_command))
    application.add_handler(CommandHandler("email", email_command))
    application.add_handler(CommandHandler("refresh", refresh_command))
    application.add_handler(CommandHandler("subjects", subjects_command))
    application.add_handler(CommandHandler("grades", grades_command))
    application.add_handler(CommandHandler("today", today_command))
    application.add_handler(CommandHandler("week", week_command))
    application.add_handler(CommandHandler("lastweek", lastweek_command))
    application.add_handler(CommandHandler("subject", subject_command))
    application.add_handler(CallbackQueryHandler(report_menu_callback,
pattern=f"^{REPORT_CALLBACK_PREFIX}"))
    application.add_handler(MessageHandler(filters.TEXT & ~filters.COMMAND,
email_message))

    schedule_jobs(application, settings)
    return application

def schedule_jobs(application: Application, settings: Settings) -> None:
    if application.job_queue is None:
        logger.warning("Job queue is unavailable. Install python-telegram-bot[job-queue].")
        return

    application.job_queue.run_daily(
        daily_summary_job,
        time=time(hour=19, minute=0, tzinfo=settings.timezone),
        days=(1, 2, 3, 4, 5),
        name="daily-summary",
    )
    application.job_queue.run_daily(
        weekly_summary_job,
        time=time(hour=10, minute=0, tzinfo=settings.timezone),
        days=(0,),
    )

```

```

        name="weekly-summary",
    )

async def start(update: Update, context: ContextTypes.DEFAULT_TYPE) -> None:
    services = get_services(context)
    chat_id = require_chat_id(update)
    user = services.storage.get(chat_id)
    if user is None:
        await update.message.reply_text("Введіть адресу електронної пошти для авторизації.")
        return

    grades = get_student_grades_cached(services, user.email)
    message = f"Ви авторизовані як {user.email}.\n\n{format_subjects(grades.disciplines)}"
    await update.message.reply_text(message, reply_markup=report_menu_keyboard())

async def stop_command(update: Update, context: ContextTypes.DEFAULT_TYPE) -> None:
    services = get_services(context)
    chat_id = require_chat_id(update)
    services.storage.delete(chat_id)
    await update.message.reply_text(
        "Діалог припинено. Ваш email видалено з бота. Щоб знову користуватися ботом, надішліть /start."
    )

async def help_command(update: Update, context: ContextTypes.DEFAULT_TYPE) -> None:
    await update.message.reply_text(f"{HELP_TEXT}\n\n{REPORT_MENU_TEXT}",
    reply_markup=report_menu_keyboard())

async def email_message(update: Update, context: ContextTypes.DEFAULT_TYPE) -> None:
    text = (update.message.text or "").strip()
    if not is_valid_email(text):
        await update.message.reply_text("Не схоже на email. Надішліть адресу у форматі student@example.com.")
        return
    await authorize_email(update, context, text)

async def email_command(update: Update, context: ContextTypes.DEFAULT_TYPE) -> None:
    if not context.args:
        await update.message.reply_text("Вкажіть email: /email student@example.com")
        return
    email = " ".join(context.args).strip()
    if not is_valid_email(email):
        await update.message.reply_text("Не схоже на email. Перевірте адресу та спробуйте ще раз.")
        return
    await authorize_email(update, context, email)

```

```

async def authorize_email(update: Update, context: ContextTypes.DEFAULT_TYPE,
email: str) -> None:
    services = get_services(context)
    chat_id = require_chat_id(update)
    normalized_email = normalize_email(email)
    services.storage.set_email(chat_id, normalized_email)
    grades = get_student_grades_cached(services, normalized_email)
    await update.message.reply_text(format_subjects(grades.disciplines),
reply_markup=report_menu_keyboard())

```

```

async def subjects_command(update: Update, context: ContextTypes.DEFAULT_TYPE) ->
None:
    user = await require_authorized_user(update, context)
    if user is None:
        return
    services = get_services(context)
    grades = get_student_grades_with_drive_refresh(services, user.email)
    await update.message.reply_text(format_subjects(grades.disciplines))

```

```

async def refresh_command(update: Update, context: ContextTypes.DEFAULT_TYPE) ->
None:
    user = await require_authorized_user(update, context)
    if user is None:
        return
    services = get_services(context)
    grades = services.repository.get_student_grades(user.email, force_reload=True)
    await update.message.reply_text(format_subjects(grades.disciplines),
reply_markup=report_menu_keyboard())

```

```

async def report_menu_callback(update: Update, context: ContextTypes.DEFAULT_TYPE)
-> None:
    query = update.callback_query
    if query is None:
        return
    await query.answer()

    services = get_services(context)
    chat_id = query.message.chat_id if query.message else require_chat_id(update)
    user = services.storage.get(chat_id)
    if user is None:
        await context.bot.send_message(
            chat_id=chat_id,
            text="Спочатку авторизуйтесь: надішліть свою email-адресу.",
        )
    return

    action = query.data.removeprefix(REPORT_CALLBACK_PREFIX) if query.data else ""
    if action == "grades":
        await send_all_grades_report_to_chat(context, chat_id, user.email)
    elif action == "today":
        await send_period_entries_to_chat(context, chat_id, user.email, "today")
    elif action == "week":
        await send_period_entries_to_chat(context, chat_id, user.email, "week")

```

```

async def grades_command(update: Update, context: ContextTypes.DEFAULT_TYPE) ->
None:
    user = await require_authorized_user(update, context)
    if user is None:
        return

    args = " ".join(context.args).strip()
    if not args:
        await send_all_grades_report(update, context, user.email)
        return

    key = args.casefold()
    if key in {"today", "сьогодні", "день"}:
        await send_period_entries(update, context, user.email, "today")
    elif key in {"week", "тиждень", "цей тиждень"}:
        await send_period_entries(update, context, user.email, "week")
    elif key in {"lastweek", "last week", "минулий тиждень", "попередній тиждень"}:
        await send_period_entries(update, context, user.email, "lastweek")
    else:
        await send_subject_entries(update, context, user.email, args)

async def today_command(update: Update, context: ContextTypes.DEFAULT_TYPE) ->
None:
    user = await require_authorized_user(update, context)
    if user:
        await send_period_entries(update, context, user.email, "today")

async def week_command(update: Update, context: ContextTypes.DEFAULT_TYPE) -> None:
    user = await require_authorized_user(update, context)
    if user:
        await send_period_entries(update, context, user.email, "week")

async def lastweek_command(update: Update, context: ContextTypes.DEFAULT_TYPE) ->
None:
    user = await require_authorized_user(update, context)
    if user:
        await send_period_entries(update, context, user.email, "lastweek")

async def subject_command(update: Update, context: ContextTypes.DEFAULT_TYPE) ->
None:
    user = await require_authorized_user(update, context)
    if user is None:
        return
    subject_query = " ".join(context.args).strip()
    if not subject_query:
        await update.message.reply_text("Вкажіть назву або частину назви
дисципліни: /subject Бази даних")
        return
    await send_subject_entries(update, context, user.email, subject_query)

async def send_entries(
    update: Update,
    context: ContextTypes.DEFAULT_TYPE,
    email: str,

```

```

        empty_text: str,
        start=None,
        end=None,
        subject_query: str | None = None,
    ) -> None:
        services = get_services(context)
        grades = get_student_grades_cached(services, email)
        entries = list(date_range_filter(list(grades.entries), start, end))

        if subject_query:
            query = subject_query.casefold()
            entries = [entry for entry in entries if query in
entry.discipline.casefold()]
            if not entries:
                empty_text = f"Записів з дисципліни «{subject_query}» не знайдено."

        text = format_entries(entries, empty_text=empty_text)
        for chunk in split_telegram_message(text):
            await update.message.reply_text(chunk, parse_mode=ParseMode.HTML)

async def send_all_grades_report(
    update: Update,
    context: ContextTypes.DEFAULT_TYPE,
    email: str,
) -> None:
    await send_all_grades_report_to_chat(context, require_chat_id(update), email)

async def send_all_grades_report_to_chat(
    context: ContextTypes.DEFAULT_TYPE,
    chat_id: int,
    email: str,
) -> None:
    services = get_services(context)
    grades = get_student_grades_cached(services, email)
    text = format_period_entries_by_date(
        list(grades.entries),
        empty_text="Оцінок і пропусків не знайдено.",
        include_discipline_averages=True,
    )
    for chunk in split_telegram_message(text):
        await context.bot.send_message(chat_id=chat_id, text=chunk,
parse_mode=ParseMode.HTML)

async def send_period_entries(
    update: Update,
    context: ContextTypes.DEFAULT_TYPE,
    email: str,
    period: str,
) -> None:
    await send_period_entries_to_chat(context, require_chat_id(update), email,
period)

async def send_period_entries_to_chat(
    context: ContextTypes.DEFAULT_TYPE,
    chat_id: int,

```

```

        email: str,
        period: str,
    ) -> None:
        services = get_services(context)
        today = today_in(services.settings.timezone)
        if period == "today":
            grades = get_student_grades_cached(services, email)
            entries = list(date_range_filter(list(grades.entries), today, today))
            text = format_today_entries(entries, grades.disciplines, today)
            for chunk in split_telegram_message(text):
                await context.bot.send_message(chat_id=chat_id, text=chunk,
                parse_mode=ParseMode.HTML)
            elif period == "week":
                start, end = current_week_bounds(today)
                await send_date_grouped_period_entries_to_chat(
                    context,
                    chat_id,
                    email,
                    start,
                    end,
                    empty_text="За поточний тиждень записів не знайдено.",
                )
            elif period == "lastweek":
                start, end = previous_week_bounds(today)
                await send_date_grouped_period_entries_to_chat(
                    context,
                    chat_id,
                    email,
                    start,
                    end,
                    empty_text="За попередній тиждень записів не знайдено.",
                )

```

```

async def send_date_grouped_period_entries(
    update: Update,
    context: ContextTypes.DEFAULT_TYPE,
    email: str,
    start,
    end,
    empty_text: str,
) -> None:
    await send_date_grouped_period_entries_to_chat(
        context,
        require_chat_id(update),
        email,
        start,
        end,
        empty_text,
    )

```

```

async def send_date_grouped_period_entries_to_chat(
    context: ContextTypes.DEFAULT_TYPE,
    chat_id: int,
    email: str,
    start,
    end,
    empty_text: str,

```

```

) -> None:
    services = get_services(context)
    grades = get_student_grades_cached(services, email)
    entries = list(date_range_filter(list(grades.entries), start, end))
    text = format_period_entries_by_date(entries, empty_text=empty_text)
    for chunk in split_telegram_message(text):
        await context.bot.send_message(chat_id=chat_id, text=chunk,
parse_mode=ParseMode.HTML)

async def send_subject_entries(
    update: Update,
    context: ContextTypes.DEFAULT_TYPE,
    email: str,
    subject_query: str,
) -> None:
    await send_entries(
        update,
        context,
        email,
        empty_text=f"Записів з дисципліни «{subject_query}» не знайдено.",
        subject_query=subject_query,
    )

async def daily_summary_job(context: ContextTypes.DEFAULT_TYPE) -> None:
    services = get_services(context)
    today = today_in(services.settings.timezone)
    await send_scheduled_daily_summary(context, today)

async def weekly_summary_job(context: ContextTypes.DEFAULT_TYPE) -> None:
    services = get_services(context)
    today = today_in(services.settings.timezone)
    start, end = current_week_bounds(today)
    await send_scheduled_summary(
        context,
        start=start,
        end=end,
        empty_text="За поточний тиждень записів не знайдено.",
    )

async def send_scheduled_summary(
    context: ContextTypes.DEFAULT_TYPE,
    start,
    end,
    empty_text: str,
) -> None:
    services = get_services(context)
    reload_repository_for_scheduled_run(services)
    for user in services.storage.all_users():
        grades = services.repository.get_student_grades(user.email)
        entries = date_range_filter(list(grades.entries), start, end)
        if not entries and not services.settings.send_empty_summaries:
            continue
        text = format_period_entries_by_date(entries, empty_text=empty_text)
        for chunk in split_telegram_message(text):
            try:

```

```

        await context.bot.send_message(
            chat_id=user.chat_id,
            text=chunk,
            parse_mode=ParseMode.HTML,
        )
    except Exception:
        logger.exception("Failed to send scheduled summary to chat_id=%s",
            user.chat_id)

async def send_scheduled_daily_summary(context: ContextTypes.DEFAULT_TYPE, today) -
> None:
    services = get_services(context)
    reload_repository_for_scheduled_run(services)
    for user in services.storage.all_users():
        grades = services.repository.get_student_grades(user.email)
        entries = date_range_filter(list(grades.entries), today, today)
        if not entries and not services.settings.send_empty_summaries:
            continue
        text = format_today_entries(entries, grades.disciplines, today)
        for chunk in split_telegram_message(text):
            try:
                await context.bot.send_message(
                    chat_id=user.chat_id,
                    text=chunk,
                    parse_mode=ParseMode.HTML,
                )
            except Exception:
                logger.exception("Failed to send scheduled daily summary to
chat_id=%s", user.chat_id)

async def require_authorized_user(update: Update, context:
ContextTypes.DEFAULT_TYPE):
    services = get_services(context)
    chat_id = require_chat_id(update)
    user = services.storage.get(chat_id)
    if user is None:
        await update.message.reply_text("Спочатку авторизуйтеся: надішліть свою
email-адресу.")
    return user

def require_chat_id(update: Update) -> int:
    if update.effective_chat is None:
        raise RuntimeError("Update has no chat.")
    return update.effective_chat.id

def get_services(context: ContextTypes.DEFAULT_TYPE) -> BotServices:
    return context.application.bot_data["services"]

def get_student_grades_cached(services: BotServices, email: str):
    return services.repository.get_student_grades(email)

def get_student_grades_with_drive_refresh(services: BotServices, email: str):
    return services.repository.get_student_grades(

```

```

        email,
        force_reload=services.settings.grades_source == "drive",
    )

```

```

def reload_repository_for_scheduled_run(services: BotServices) -> None:
    if services.settings.grades_source == "drive":
        services.repository.reload()

```

1.6. eu_grades_bot/config.py

```

from __future__ import annotations

import os
from dataclasses import dataclass
from pathlib import Path
from zoneinfo import ZoneInfo

from dotenv import load_dotenv

@dataclass(frozen=True)
class Settings:
    telegram_bot_token: str
    grades_source: str
    local_grades_dir: Path
    drive_folder_id: str | None
    google_credentials_file: Path | None
    timezone: ZoneInfo
    data_dir: Path
    cache_ttl_seconds: int
    send_empty_summaries: bool

def _bool_from_env(name: str, default: bool) -> bool:
    raw = os.getenv(name)
    if raw is None:
        return default
    return raw.strip().lower() in {"1", "true", "yes", "y", "on"}

def load_settings() -> Settings:
    load_dotenv()

    token = os.getenv("TELEGRAM_BOT_TOKEN", "").strip()
    source = os.getenv("GRADES_SOURCE", "local").strip().lower()
    if source not in {"local", "drive"}:
        raise ValueError("GRADES_SOURCE must be either 'local' or 'drive'.")

    local_dir = Path(os.getenv("GRADES_LOCAL_DIR", "examples")).expanduser()
    drive_folder_id = os.getenv("GRADES_DRIVE_FOLDER_ID", "").strip() or None
    credentials = os.getenv("GOOGLE_APPLICATION_CREDENTIALS", "").strip()
    credentials_path = Path(credentials).expanduser() if credentials else None
    timezone = ZoneInfo(os.getenv("BOT_TIMEZONE", "Europe/Kyiv"))
    data_dir = Path(os.getenv("DATA_DIR", ".bot_data")).expanduser()

    return Settings(
        telegram_bot_token=token,

```

```

grades_source=source,
local_grades_dir=local_dir,
drive_folder_id=drive_folder_id,
google_credentials_file=credentials_path,
timezone=timezone,
data_dir=data_dir,
cache_ttl_seconds=int(os.getenv("CACHE_TTL_SECONDS", "300")),
send_empty_summaries=_bool_from_env("SEND_EMPTY_SUMMARIES", True),
)

```

1.7. eu_grades_bot/dates.py

```

from __future__ import annotations

from datetime import date, datetime, timedelta
from zoneinfo import ZoneInfo

def today_in(timezone: ZoneInfo) -> date:
    return datetime.now(timezone).date()

def current_week_bounds(anchor: date) -> tuple[date, date]:
    start = anchor - timedelta(days=anchor.weekday())
    return start, start + timedelta(days=6)

def previous_week_bounds(anchor: date) -> tuple[date, date]:
    current_start, _ = current_week_bounds(anchor)
    previous_start = current_start - timedelta(days=7)
    return previous_start, previous_start + timedelta(days=6)

def date_range_filter(
    entries: list,
    start: date | None = None,
    end: date | None = None,
):
    if start is None and end is None:
        return entries
    return [
        entry
        for entry in entries
        if (start is None or entry.date >= start) and (end is None or entry.date <=
end)
    ]

```

1.8. eu_grades_bot/drive.py

```

from __future__ import annotations

import io
import re
from dataclasses import dataclass
import json
import logging
from pathlib import Path

```

```

GOOGLE_SHEETS_MIME = "application/vnd.google-apps.spreadsheet"
GOOGLE_SHORTCUT_MIME = "application/vnd.google-apps.shortcut"
XLSX_MIME = "application/vnd.openxmlformats-officedocument.spreadsheetml.sheet"
SUPPORTED_DRIVE_WORKBOOK_MIMES = {GOOGLE_SHEETS_MIME, XLSX_MIME}
logger = logging.getLogger(__name__)

@dataclass(frozen=True)
class WorkbookSource:
    title: str
    path: Path

class LocalWorkbookProvider:
    def __init__(
        self,
        folder: Path,
        credentials_file: Path | None = None,
        cache_dir: Path | None = None,
    ):
        self.folder = folder
        self.credentials_file = credentials_file
        self.cache_dir = cache_dir

    def list_workbooks(self) -> list[WorkbookSource]:
        if not self.folder.exists():
            raise FileNotFoundError(f"Grades folder does not exist: {self.folder}")
        workbook_files = sorted(
            [
                *self.folder.glob("*.xlsx"),
                *self.folder.glob("*.xlsm"),
            ],
            key=lambda p: p.name.lower(),
        )
        workbooks = [
            WorkbookSource(title=path.stem, path=path)
            for path in workbook_files
            if not path.name.startswith("~$")
        ]

        gsheets_files = sorted(self.folder.glob("*.gsheet"), key=lambda p:
p.name.lower())
        if gsheets_files:
            workbooks.extend(self._export_gsheet_files(gsheets_files))

        return workbooks

    def _export_gsheet_files(self, paths: list[Path]) -> list[WorkbookSource]:
        if not self.credentials_file:
            raise RuntimeError(
                "Local Google Drive .gsheet files are shortcuts, not spreadsheets.
                "
                "Set GOOGLE_APPLICATION_CREDENTIALS so the bot can export them, "
                "or use GRADES_SOURCE=drive."
            )
        cache_dir = self.cache_dir or (self.folder / ".gsheet_cache")
        cache_dir.mkdir(parents=True, exist_ok=True)

```

```

service = build_drive_service(self.credentials_file)

workbooks: list[WorkbookSource] = []
for path in paths:
    spreadsheet_id = extract_google_sheet_id_from_gsheet(path)
    if not spreadsheet_id:
        raise ValueError(f"Could not read Google Sheets id from {path}")

    safe_name = DriveWorkbookProvider._safe_filename(path.stem)
    target = cache_dir / f"{spreadsheet_id}-{safe_name}.xlsx"
    request = service.files().export_media(fileId=spreadsheet_id,
mimeType=XLXS_MIME)
    download_media(request, target)
    workbooks.append(WorkbookSource(title=path.stem, path=target))

return workbooks

class DriveWorkbookProvider:
    def __init__(self, folder_id: str, credentials_file: Path, cache_dir: Path):
        self.folder_id = normalize_drive_id(folder_id)
        self.credentials_file = credentials_file
        self.cache_dir = cache_dir

    def list_workbooks(self) -> list[WorkbookSource]:
        service = self._build_service()
        self.cache_dir.mkdir(parents=True, exist_ok=True)
        files = self._list_drive_files(service)
        logger.info("Drive folder %s returned %d file(s).", self.folder_id,
len(files))
        workbooks: list[WorkbookSource] = []

        for file_info in files:
            if is_gsheet_metadata_file(file_info):
                workbooks.append(self._export_drive_gsheet_metadata_file(service,
file_info))
                continue

            workbook_file = self._resolve_workbook_file(file_info)
            if workbook_file is None:
                continue

            file_id = workbook_file["id"]
            name = workbook_file["name"]
            mime_type = workbook_file.get("mimeType")
            safe_name = self._safe_filename(name)
            if not safe_name.lower().endswith(".xlsx"):
                safe_name = f"{safe_name}.xlsx"
            target = self.cache_dir / f"{file_id}-{safe_name}"

            if mime_type == GOOGLE_SHEETS_MIME:
                request = service.files().export_media(fileId=file_id,
mimeType=XLXS_MIME)
            else:
                request = service.files().get_media(fileId=file_id)

            self._download(request, target)
            workbooks.append(WorkbookSource(title=Path(name).stem, path=target))

```

```

        logger.info("Drive folder %s produced %d workbook(s).", self.folder_id,
len(workbooks))
        return workbooks

    def _export_drive_gsheet_metadata_file(self, service, file_info: dict) ->
WorkbookSource:
        file_id = file_info["id"]
        name = file_info.get("name", "workbook.gsheet")
        safe_name = self._safe_filename(name)
        metadata_target = self.cache_dir / f"{file_id}-{safe_name}"
        metadata_request = service.files().get_media(fileId=file_id)
        self._download(metadata_request, metadata_target)

        spreadsheet_id = extract_google_sheet_id_from_gsheet(metadata_target)
        if not spreadsheet_id:
            raise ValueError(f"Could not read Google Sheets id from Drive .gsheet
file: {name}")

        xlsx_target = self.cache_dir / f"{spreadsheet_id}-
{self._safe_filename(Path(name).stem)}.xlsx"
        export_request = service.files().export_media(fileId=spreadsheet_id,
mimeType=EXLSX_MIME)
        self._download(export_request, xlsx_target)
        return WorkbookSource(title=Path(name).stem, path=xlsx_target)

    def _build_service(self):
        return build_drive_service(self.credentials_file)

    def _list_drive_files(self, service) -> list[dict]:
        query = f"'{self.folder_id}' in parents and trashed = false and mimeType !=
'application/vnd.google-apps.folder'"
        result: list[dict] = []
        page_token = None
        while True:
            response = (
                service.files()
                .list(
                    q=query,
                    corpora="allDrives",
                    spaces="drive",
                    fields=(
                        "nextPageToken, "
                        "files(id, name, mimeType, modifiedTime, "
                        "shortcutDetails(targetId,targetMimeType)) "
                    ),
                    pageToken=page_token,
                    pageSize=1000,
                    supportsAllDrives=True,
                    includeItemsFromAllDrives=True,
                )
                .execute()
            )
            result.extend(response.get("files", []))
            page_token = response.get("nextPageToken")
            if not page_token:
                return result

    @staticmethod
    def _resolve_workbook_file(file_info: dict) -> dict | None:

```

```

mime_type = file_info.get("mimeType")
if mime_type in SUPPORTED_DRIVE_WORKBOOK_MIMES:
    return file_info

if mime_type != GOOGLE_SHORTCUT_MIME:
    return None

shortcut_details = file_info.get("shortcutDetails") or {}
target_mime_type = shortcut_details.get("targetMimeType")
target_id = shortcut_details.get("targetId")
if target_mime_type not in SUPPORTED_DRIVE_WORKBOOK_MIMES or not target_id:
    return None

return {
    "id": target_id,
    "name": file_info.get("name", "workbook"),
    "mimeType": target_mime_type,
}

@staticmethod
def _download(request, target: Path) -> None:
    download_media(request, target)

@staticmethod
def _safe_filename(name: str) -> str:
    cleaned = re.sub(r'[\<:\\"/\\\\\|?*\]+' , "_", name).strip()
    return cleaned or "workbook"

def normalize_drive_id(value: str) -> str:
    value = value.strip()
    folder_match = re.search(r"/folders/([^\/?#]+)", value)
    if folder_match:
        return folder_match.group(1)
    id_match = re.search(r"[?&]id=([^\&#]+)", value)
    if id_match:
        return id_match.group(1)
    spreadsheet_match = re.search(r"/spreadsheets/d/([^\/?#]+)", value)
    if spreadsheet_match:
        return spreadsheet_match.group(1)
    return value.rstrip("/")

def is_gsheel_metadata_file(file_info: dict) -> bool:
    name = str(file_info.get("name", "")).casefold()
    mime_type = file_info.get("mimeType")
    return name.endswith(".gsheet") and mime_type not in {
        GOOGLE_SHEETS_MIME,
        GOOGLE_SHORTCUT_MIME,
    }

def extract_google_sheet_id_from_gsheel(path: Path) -> str | None:
    text = path.read_text(encoding="utf-8-sig")
    try:
        payload = json.loads(text)
    except json.JSONDecodeError:
        payload = {}

```

```

if isinstance(payload, dict):
    for key in ("doc_id", "id"):
        value = payload.get(key)
        if isinstance(value, str) and value.strip():
            return normalize_drive_id(value)

    resource_id = payload.get("resource_id")
    if isinstance(resource_id, str) and ":" in resource_id:
        candidate = resource_id.rsplit(":", 1)[-1].strip()
        if candidate:
            return candidate

    url = payload.get("url")
    if isinstance(url, str) and url.strip():
        return normalize_drive_id(url)

normalized_text = text.replace("\\\\", "/")
spreadsheet_match = re.search(r"/spreadsheets/d/([^\?#\\"']+)", normalized_text)
if spreadsheet_match:
    return spreadsheet_match.group(1)

doc_id_match = re.search(r'"doc_id"\s*:\s*"([^\"]+)"', normalized_text)
if doc_id_match:
    return normalize_drive_id(doc_id_match.group(1))

resource_match = re.search(r'"resource_id"\s*:\s*"([^\"]+):([^\"]+)"',
normalized_text)
if resource_match:
    return resource_match.group(1)

return None

def build_drive_service(credentials_file: Path):
    try:
        from google.oauth2 import service_account
        from googleapiclient.discovery import build
    except ImportError as exc:
        raise RuntimeError(
            "Google Drive export requires google-api-python-client and google-
auth."
        ) from exc

    scopes = ["https://www.googleapis.com/auth/drive.readonly"]
    credentials = service_account.Credentials.from_service_account_file(
        credentials_file,
        scopes=scopes,
    )
    return build("drive", "v3", credentials=credentials, cache_discovery=False)

def download_media(request, target: Path) -> None:
    from googleapiclient.http import MediaIoBaseDownload

    buffer = io.BytesIO()
    downloader = MediaIoBaseDownload(buffer, request)
    done = False
    while not done:
        _, done = downloader.next_chunk()

```

```
target.write_bytes(buffer.getvalue())
```

1.9. eu_grades_bot/formatting.py

```
from __future__ import annotations

import html
from collections import defaultdict
from datetime import date

from .grades import GradeEntry, is_absence, numeric_grade

LOW_MARKER = "🔴"
HIGH_MARKER = "🟢"
DEFAULT_MARKER = "•"
MAX_TELEGRAM_MESSAGE_LENGTH = 3900

def format_subjects(disciplines: tuple[str, ...]) -> str:
    if not disciplines:
        return "За цією email-адресою дисципліни не знайдено."
    lines = ["Знайдено дисципліни:"]
    lines.extend(f"• {html.escape(discipline)}" for discipline in disciplines)
    return "\n".join(lines)

def format_entries(entries: list[GradeEntry] | tuple[GradeEntry, ...], empty_text:
str) -> str:
    if not entries:
        return empty_text

    grouped: dict[str, list[GradeEntry]] = defaultdict(list)
    for entry in entries:
        grouped[entry.discipline].append(entry)

    parts: list[str] = []
    for discipline, discipline_entries in grouped.items():
        parts.append(f"<b>{html.escape(discipline)}</b>")
        for entry in sorted(discipline_entries, key=lambda e: (e.date,
e.column_index, e.group.casefold())):
            parts.append(format_entry_line(entry))
        parts.append("")
    return "\n".join(parts).strip()

def format_period_entries_by_date(
    entries: list[GradeEntry] | tuple[GradeEntry, ...],
    empty_text: str,
    include_discipline_averages: bool = False,
) -> str:
    if not entries:
        return empty_text

    grouped: dict[date, list[GradeEntry]] = defaultdict(list)
    for entry in entries:
        grouped[entry.date].append(entry)
```

```

parts: list[str] = []
for day in sorted(grouped):
    parts.append(format_date(day))
    for entry in sorted(grouped[day], key=lambda e: (e.discipline.casefold(),
e.column_index, e.group.casefold())):
        parts.append(format_period_entry_line(entry))
    parts.append("")

parts.append(format_period_summary(entries))
if include_discipline_averages:
    parts.extend(["", format_discipline_averages(entries)])
return "\n".join(parts).strip()

def format_period_entry_line(entry: GradeEntry) -> str:
    marker = marker_for_value(entry.value)
    return f"{marker} {html.escape(entry.discipline)} - {html.escape(display_value(entry.value))}"

def format_period_summary(entries: list[GradeEntry] | tuple[GradeEntry, ...]) -> str:
    total_lessons = len(entries)
    absence_count = sum(1 for entry in entries if is_absence(entry.value))
    days_count = len({entry.date for entry in entries})
    absence_percent = round(absence_count / total_lessons * 100) if total_lessons
else 0
    grades = [grade for entry in entries if (grade := numeric_grade(entry.value))
is not None]
    excellent_count = sum(1 for grade in grades if grade > 9)
    good_count = sum(1 for grade in grades if 7 <= grade <= 9)
    satisfactory_count = sum(1 for grade in grades if 5 <= grade <= 7)
    unsatisfactory_count = sum(1 for grade in grades if grade < 5)

    return "\n".join(
        [
            f"Загальна кількість пропусків: {absence_count} з {total_lessons}. ",
            f"Пропущено {absence_percent}% занять за {days_count}
{day_word(days_count)}. ",
            f"Загальна кількість оцінок: {len(grades)}. ",
            f"Оцінки \"відмінно\": {excellent_count}. ",
            f"Оцінки \"добре\": {good_count}. ",
            f"Оцінки \"задовільно\": {satisfactory_count}. ",
            f"Оцінки \"незадовільно\": {unsatisfactory_count}. ",
        ]
    )

def format_discipline_averages(entries: list[GradeEntry] | tuple[GradeEntry, ...])
-> str:
    disciplines = sorted({entry.discipline for entry in entries}, key=str.casefold)
    grades_by_discipline: dict[str, list[int]] = defaultdict(list)
    for entry in entries:
        grade = numeric_grade(entry.value)
        if grade is not None:
            grades_by_discipline[entry.discipline].append(grade)

    lines = ["Середній бал за дисциплінами:"]
    for discipline in disciplines:

```

```

        grades = grades_by_discipline.get(discipline, [])
        if grades:
            average = sum(grades) / len(grades)
            lines.append(f"{html.escape(discipline)} - {average:.1f}")
        else:
            lines.append(f"{html.escape(discipline)} - немає оцінок")
    return "\n".join(lines)

def day_word(count: int) -> str:
    if count % 10 == 1 and count % 100 != 11:
        return "день"
    if count % 10 in {2, 3, 4} and count % 100 not in {12, 13, 14}:
        return "дні"
    return "днів"

def format_today_entries(
    entries: list[GradeEntry] | tuple[GradeEntry, ...],
    disciplines: tuple[str, ...],
    day: date,
) -> str:
    grades = [entry for entry in entries if not is_absence(entry.value)]
    absences = [entry for entry in entries if is_absence(entry.value)]
    subject_list = format_subject_list(disciplines)

    parts: list[str] = [format_date(day), ""]
    if not grades and not absences:
        parts.extend([subject_list, "Не має даних щодо пропусків та оцінок"])
        return "\n".join(parts).strip()

    if grades:
        parts.append("Оцінки:")
        parts.extend(format_today_entry_line(entry) for entry in
sort_today_entries(grades))
    else:
        parts.extend([subject_list, "Оцінки відсутні"])

    parts.append("")
    if absences:
        parts.append("Пропуски:")
        parts.extend(format_today_absence_line(entry) for entry in
sort_today_entries(absences))
        parts.append(f"Загальна кількість пропусків: {len(absences)}")
    else:
        parts.extend([subject_list, "Пропуски відсутні"])

    return "\n".join(parts).strip()

def format_subject_list(disciplines: tuple[str, ...]) -> str:
    if not disciplines:
        return "Дисципліни не знайдено."
    return "\n".join(html.escape(discipline) for discipline in disciplines)

def sort_today_entries(entries: list[GradeEntry]) -> list[GradeEntry]:
    return sorted(entries, key=lambda e: (e.discipline.casefold(), e.column_index,
e.group.casefold()))

```

```

def format_today_entry_line(entry: GradeEntry) -> str:
    return f"{html.escape(entry.discipline)} - {html.escape(display_value(entry.value))}"

def format_today_absence_line(entry: GradeEntry) -> str:
    return html.escape(entry.discipline)

def format_entry_line(entry: GradeEntry) -> str:
    marker = marker_for_value(entry.value)
    return f"{marker} {format_date(entry.date)} - {html.escape(display_value(entry.value))}"

def display_value(value: str) -> str:
    if is_absence(value):
        return "відсутній"
    return value

def marker_for_value(value: str) -> str:
    if is_absence(value):
        return LOW_MARKER
    grade = numeric_grade(value)
    if grade is None:
        return DEFAULT_MARKER
    if 1 <= grade <= 3:
        return LOW_MARKER
    if 10 <= grade <= 12:
        return HIGH_MARKER
    return DEFAULT_MARKER

def format_date(value: date) -> str:
    return value.strftime("%d.%m.%Y")

def split_telegram_message(text: str) -> list[str]:
    if len(text) <= MAX_TELEGRAM_MESSAGE_LENGTH:
        return [text]

    chunks: list[str] = []
    current: list[str] = []
    current_length = 0
    for paragraph in text.split("\n\n"):
        paragraph_length = len(paragraph) + 2
        if current and current_length + paragraph_length > MAX_TELEGRAM_MESSAGE_LENGTH:
            chunks.append("\n\n".join(current))
            current = [paragraph]
            current_length = paragraph_length
        else:
            current.append(paragraph)
            current_length += paragraph_length
    if current:
        chunks.append("\n\n".join(current))

```

```
return chunks
```

1.10. eu_grades_bot/grades.py

```
from __future__ import annotations

import re
import time
import logging
from collections import defaultdict
from dataclasses import dataclass
from datetime import date, datetime
from pathlib import Path
from typing import Iterable, Protocol

from openpyxl import load_workbook
from openpyxl.worksheet.worksheet import Worksheet

from .drive import WorkbookSource

EMAIL_RE = re.compile(r"^[^@\s]+@[^@\s]+\.[^@\s]+$")
logger = logging.getLogger(__name__)

@dataclass(frozen=True)
class GradeEntry:
    discipline: str
    group: str
    student_name: str
    email: str
    date: date
    value: str
    source_file: str
    column_index: int

@dataclass(frozen=True)
class StudentGrades:
    email: str
    student_name: str | None
    disciplines: tuple[str, ...]
    entries: tuple[GradeEntry, ...]

class WorkbookProvider(Protocol):
    def list_workbooks(self) -> list[WorkbookSource]:
        ...

class GradesRepository:
    def __init__(self, provider: WorkbookProvider, cache_ttl_seconds: int = 300):
        self.provider = provider
        self.cache_ttl_seconds = cache_ttl_seconds
        self._loaded_at = 0.0
        self._entries_by_email: dict[str, list[GradeEntry]] = {}
        self._names_by_email: dict[str, str] = {}
```

```

def get_student_grades(self, email: str, force_reload: bool = False) ->
StudentGrades:
    normalized_email = normalize_email(email)
    self._ensure_loaded(force_reload=force_reload)
    entries = tuple(sorted(self._entries_by_email.get(normalized_email, []),
key=_entry_sort_key))
    disciplines = tuple(dict.fromkeys(entry.discipline for entry in entries))
    return StudentGrades(
        email=normalized_email,
        student_name=self._names_by_email.get(normalized_email),
        disciplines=disciplines,
        entries=entries,
    )

def _ensure_loaded(self, force_reload: bool = False) -> None:
    if (
        not force_reload
        and self._loaded_at > 0
        and time.monotonic() - self._loaded_at < self.cache_ttl_seconds
    ):
        return
    self.reload()

def reload(self) -> None:
    entries_by_email: dict[str, list[GradeEntry]] = defaultdict(list)
    names_by_email: dict[str, str] = {}

    sources = self.provider.list_workbooks()
    logger.info("Loading grades from %d workbook(s).", len(sources))
    total_entries = 0
    for source in sources:
        discipline = discipline_from_title(source.title)
        source_entries = 0
        for entry in iter_workbook_entries(source.path, discipline,
source.title):
            entries_by_email[entry.email].append(entry)
            names_by_email.setdefault(entry.email, entry.student_name)
            source_entries += 1
        total_entries += source_entries
        logger.info("Workbook %s produced %d grade/absence record(s).",
source.title, source_entries)

    self._entries_by_email = dict(entries_by_email)
    self._names_by_email = names_by_email
    self._loaded_at = time.monotonic()
    logger.info(
        "Loaded %d grade/absence record(s) for %d email(s).",
        total_entries,
        len(self._entries_by_email),
    )

def normalize_email(email: str) -> str:
    return email.strip().lower()

def is_valid_email(email: str) -> bool:
    return bool(EMAIL_RE.match(normalize_email(email)))

```

```

def discipline_from_title(title: str) -> str:
    stem = Path(title).stem
    if "-" in stem:
        return stem.rsplit("-", 1)[0].strip()
    return stem.strip()

def iter_workbook_entries(path: Path, discipline: str, source_title: str | None =
None) -> Iterable[GradeEntry]:
    workbook = load_workbook(path, data_only=True, read_only=False)
    try:
        for sheet in workbook.worksheets:
            yield from iter_sheet_entries(sheet, discipline, source_title or
path.name)
    finally:
        workbook.close()

def iter_sheet_entries(sheet: Worksheet, discipline: str, source_file: str) ->
Iterable[GradeEntry]:
    header_row = _find_header_row(sheet)
    if header_row is None:
        return

    email_col = _find_email_column(sheet, header_row)
    if email_col is None:
        return

    name_col = _find_name_column(sheet, header_row)
    date_columns = _find_date_columns(sheet, header_row)
    if not date_columns:
        return

    for row_index in range(header_row + 1, sheet.max_row + 1):
        raw_email = sheet.cell(row=row_index, column=email_col).value
        if raw_email is None:
            continue
        email = normalize_email(str(raw_email))
        if not is_valid_email(email):
            continue

        raw_name = sheet.cell(row=row_index, column=name_col).value if name_col
else None
        student_name = str(raw_name).strip() if raw_name else ""

        for column_index, entry_date in date_columns:
            raw_value = sheet.cell(row=row_index, column=column_index).value
            value = normalize_grade_value(raw_value)
            if value is None:
                continue
            yield GradeEntry(
                discipline=discipline,
                group=sheet.title,
                student_name=student_name,
                email=email,
                date=entry_date,
                value=value,
                source_file=source_file,

```

```

        column_index=column_index,
    )

def normalize_grade_value(raw_value) -> str | None:
    if raw_value is None:
        return None
    if isinstance(raw_value, str):
        value = raw_value.strip()
        if not value:
            return None
        if value.lower() == "B":
            return "B"
        return value
    if isinstance(raw_value, float) and raw_value.is_integer():
        return str(int(raw_value))
    if isinstance(raw_value, int):
        return str(raw_value)
    return str(raw_value).strip() or None

def is_absence(value: str) -> bool:
    return value.strip().lower() == "B"

def numeric_grade(value: str) -> int | None:
    try:
        number = float(value.replace(",", "."))
    except ValueError:
        return None
    if not number.is_integer():
        return None
    return int(number)

def _find_header_row(sheet: Worksheet) -> int | None:
    for row_index in range(1, min(sheet.max_row, 10) + 1):
        if _find_email_column(sheet, row_index) is not None:
            return row_index
    return None

def _find_email_column(sheet: Worksheet, row_index: int) -> int | None:
    for column_index in range(1, sheet.max_column + 1):
        value = sheet.cell(row=row_index, column=column_index).value
        text = str(value).strip().lower() if value is not None else ""
        if "електрон" in text or "email" in text or "e-mail" in text:
            return column_index
    return None

def _find_name_column(sheet: Worksheet, row_index: int) -> int | None:
    for column_index in range(1, sheet.max_column + 1):
        value = sheet.cell(row=row_index, column=column_index).value
        text = str(value).strip().lower() if value is not None else ""
        if "прізвище" in text or "піб" in text or "студент" in text:
            return column_index
    return None

```

```

def _find_date_columns(sheet: Worksheet, row_index: int) -> list[tuple[int, date]]:
    result: list[tuple[int, date]] = []
    for column_index in range(1, sheet.max_column + 1):
        value = sheet.cell(row=row_index, column=column_index).value
        parsed = parse_date_cell(value)
        if parsed is not None:
            result.append((column_index, parsed))
    return result

def parse_date_cell(value) -> date | None:
    if isinstance(value, datetime):
        return value.date()
    if isinstance(value, date):
        return value
    if not isinstance(value, str):
        return None

    text = value.strip()
    if not text:
        return None
    for fmt in ("%d.%m.%Y", "%d.%m.%y", "%Y-%m-%d", "%d/%m/%Y", "%d/%m/%y"):
        try:
            return datetime.strptime(text, fmt).date()
        except ValueError:
            continue
    return None

def _entry_sort_key(entry: GradeEntry):
    return (entry.discipline.casefold(), entry.date, entry.column_index,
    entry.group.casefold())

```

1.11. eu_grades_bot/storage.py

```

from __future__ import annotations

import json
from dataclasses import dataclass
from datetime import datetime, timezone
from pathlib import Path

@dataclass(frozen=True)
class AuthorizedUser:
    chat_id: int
    email: str
    created_at: str
    updated_at: str

class UserStorage:
    def __init__(self, path: Path):
        self.path = path

    def get(self, chat_id: int) -> AuthorizedUser | None:
        payload = self._load()

```

```

raw = payload.get(str(chat_id))
if not raw:
    return None
return AuthorizedUser(
    chat_id=chat_id,
    email=raw["email"],
    created_at=raw.get("created_at", ""),
    updated_at=raw.get("updated_at", ""),
)

def set_email(self, chat_id: int, email: str) -> AuthorizedUser:
    payload = self._load()
    now = datetime.now(timezone.utc).isoformat()
    previous = payload.get(str(chat_id), {})
    payload[str(chat_id)] = {
        "email": email,
        "created_at": previous.get("created_at", now),
        "updated_at": now,
    }
    self._save(payload)
    return self.get(chat_id) # type: ignore[return-value]

def delete(self, chat_id: int) -> bool:
    payload = self._load()
    existed = str(chat_id) in payload
    payload.pop(str(chat_id), None)
    self._save(payload)
    return existed

def all_users(self) -> list[AuthorizedUser]:
    payload = self._load()
    users: list[AuthorizedUser] = []
    for raw_chat_id, raw in payload.items():
        try:
            chat_id = int(raw_chat_id)
        except ValueError:
            continue
        users.append(
            AuthorizedUser(
                chat_id=chat_id,
                email=raw["email"],
                created_at=raw.get("created_at", ""),
                updated_at=raw.get("updated_at", ""),
            )
        )
    return users

def _load(self) -> dict:
    if not self.path.exists():
        return {}
    with self.path.open("r", encoding="utf-8") as handle:
        return json.load(handle)

def _save(self, payload: dict) -> None:
    self.path.parent.mkdir(parents=True, exist_ok=True)
    temporary = self.path.with_suffix(".tmp")
    with temporary.open("w", encoding="utf-8") as handle:
        json.dump(payload, handle, ensure_ascii=False, indent=2)
    temporary.replace(self.path)

```